Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 4:

The xv6 Shell from the Outside

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Reițman Patricia

Chișinău – 2026

Contents

1 Lab 4: Track B – The xv6 Shell from the Outside **2**
1.1 Before you start . . . 2
1.2 Part 1. Fix sleep for the current xv6 . . . 2
1.2.1 System Call Verification in user.h . . . 2
1.2.2 Implementation of user/sleep.c . . . 2
1.2.3 Integration into Makefile . . . 3
1.2.4 Build and Testing in QEMU . . . 3
1.3 Part 2. bash versus the xv6 shell . . . 3
1.3.1 Comparative Test Suite . . . 3
1.3.2 Terminal Execution Record: GNU bash . . . 4
1.3.3 Terminal Execution Record: xv6 shell . . . 5
1.4 Part 3. Who expands *? A program that prints its argv . . . 6
1.4.1 Utility Implementation for xv6: user/args.c . . . 6
1.4.2 Utility Implementation for Linux: args_linux.c . . . 7
1.4.3 Side-by-Side Execution . . . 7
1.5 Part 4. Read the shell . . . 9
1.5.1 Source Code Inspection of user/sh.c . . . 9
1.5.2 Architecture of the xv6 Command Interpreter . . . 10

2 Conclusion **11**

1 Lab 4: Track B – The xv6 Shell from the Outside

1.1 Before you start

Purpose and Scope

The entire user interface of the educational UNIX operating system xv6 is implemented in a single user-space program, `user/sh.c`, spanning approximately 500 lines of concise C code. The primary purpose of this laboratory work is to investigate the mechanics of command-line interpretation by running the xv6 shell side-by-side with modern GNU `bash` on Linux.

Through experimental side-by-side comparison, we dissect what capabilities an operating system shell is fundamentally required to provide (process invocation, file descriptor redirection, and inter-process pipelining) versus what conveniences modern shells like `bash` layer on top (pathname globbing, environment variable expansion, quotation parsing, and interactive line-editing). Furthermore, by writing a dedicated argument-inspection utility (`args`) in both environments, we empirically prove where the boundary of responsibility lies between user-space utilities, the shell interpreter, and the kernel.

Environment Setup and Identification

```
➜  ~ cd ~/xv6-riscv
➜  xv6-riscv git:(riscv) whoami
ionut
➜  xv6-riscv git:(riscv) uname -a
Linux arch 7.2.8-1-cachyos #1 SMP PREEMPT_DYNAMIC Sat, 26 Sep 2026 19:43:45 +0000
    x86_64 GNU/Linux
➜  xv6-riscv git:(riscv) git status
On branch riscv
Your branch is up to date with 'origin/riscv'.
```

1.2 Part 1. Fix sleep for the current xv6

1.2.1 System Call Verification in user.h

In August 2025, the xv6 upstream development team renamed the timer system call `sleep()` to `pause()` to avoid conceptual naming collisions with POSIX sleep semantics. We inspect `user/user.h` to determine which system call declaration is present:

```
➜  xv6-riscv git:(riscv) grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

The header file explicitly declares `int pause(int);`. Consequently, our user-space utility must invoke `pause()` to enter the sleep state.

1.2.2 Implementation of user/sleep.c

We implement `user/sleep.c`, ensuring proper argument validation and usage diagnostics:

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
```

```
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel (was sleep() before Aug
    2025)
  exit(0);
}
```

1.2.3 Integration into Makefile

We ensure that the binary target `$U/_sleep\` is registered within the `UPROGS` definition list in the xv6 build configuration:

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
        $U/_forktest\
        ...
```

1.2.4 Build and Testing in QEMU

We compile the operating system and user-space binaries and boot the kernel inside QEMU:

```
➜  xv6-riscv git:(riscv) make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,
    format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

The tests succeed: `sleep 20` pauses the shell for 20 timer ticks before re-prompting, and invoking `sleep` without arguments triggers the error message on file descriptor 2 with exit code 1.

1.3 Part 2. bash versus the xv6 shell

1.3.1 Comparative Test Suite

We execute a standardized suite of 10 commands in both shells:

- **Linux bash:** Executed inside an isolated directory `~/shtest` containing an empty file named `README`.
- **xv6 shell:** Executed at the xv6 interactive prompt `$` inside QEMU.

Command	Tests	bash (Linux)	xv6 shell	Why / Explanation
`echo hello > f ; cat < f`	Redirection	`hello`	`hello`	Both shells redirect stdout (>) and stdin (<) using OS file descriptors.
`ls | grep READ`	Pipeline	`README`	`README 2 2 2441`	Both support pipes (|). xv6 `ls` includes type, inode, and byte size.
`(echo a; echo b) | wc`	Grouping, list	`2 2 4`	`2 2 4`	Both shells parse parentheses (`(...)`) as command blocks whose combined stream is piped.
`echo one ; echo two`	Sequence	`one` `two`	`one` `two`	Both shells use ; to execute commands sequentially.
`echo bg &`	Background	`bg` (asynchronous)	`bg` (asynchronous)	Both support & by forking without calling `wait()` in the parent.
`ls *`	Globbing	`args args_linux.c f README`	`ls: cannot open *`	bash expands * to matching filenames; xv6 passes * literally.
`echo $HOME`	Variables	`/home/ionut`	`$HOME`	bash expands environment variables; xv6 has no variables and prints `$HOME` literally.
`echo "a b"`	Quotes	`a b`	`"a b"`	bash removes quotes and groups words; xv6 treats quotes as normal characters.
`mkdir d ; cd d ; ls ..`	PATH lookup	`args ... README`	`exec ls failed`	bash resolves `ls` via `$PATH` (`/usr/bin`); xv6 only checks current directory `/d`.
`Arrow Up, Tab, Ctrl+R`	History / Completion	Recalls command, autocompletes, searches	Prints escape codes / tabs; no history	bash links against GNU Readline; xv6 reads raw characters via `gets()`.

Table 1: Comprehensive behavioral comparison between GNU bash and xv6 sh

1.3.2 Terminal Execution Record: GNU bash

```
➜  ~ mkdir -p ~/shtest && cd ~/shtest && touch README
➜  shtest echo hello > f ; cat < f
hello
➜  shtest ls | grep READ
README
➜  shtest (echo a; echo b) | wc
      2       2       4
➜  shtest echo one ; echo two
one
two
➜  shtest echo bg &
[1] 23412
bg
➜  shtest ls *
```

```
args  args_linux.c  f  README
➜  shtest echo $HOME
/home/ionut
➜  shtest echo "a b"
a b
➜  shtest mkdir d ; cd d ; ls ..
args  args_linux.c  d  f  README
➜  shtest cd ~/shtest
```

1.3.3 Terminal Execution Record: xv6 shell

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
exec ls failed
$ /ls ..
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35864
echo            2 4 34760
...
d               1 24 32
f               2 25 6
$
```

Observe:

1. **List the features both shells have, and the ones only bash has.**
   - **Features common to both shells:**
     - Input, output, and appending redirection (<, >, >>)
     - Inter-process communication pipelines (|)
     - Command sequencing via semicolons (;)
     - Command grouping in subshells ((...))
     - Asynchronous background process execution (&)
     - Built-in directory navigation (cd)
   - **Features present only in bash:**

- `$PATH` binary resolution across system directories
- Pathname expansion and wildcard globbing (`*`, `?`, `[...]`)
- Environment and shell variables (`$HOME`, `$USER`, `$?`)
- Quote parsing, parameter grouping, and escape sequences (`""`, `''`, \)
- Command-line editing, history browsing, reverse search, and Tab completion (GNU Readline)
- Control-flow statements (`if`, `for`, `while`) and command substitution (`$(...)`)

2. **In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway.**
   Why it fails: The xv6 shell does not maintain or search a `PATH` environment variable. When a command such as `ls` is executed without a leading slash, the shell directly invokes the system call `exec("ls", ...)`. The xv6 kernel attempts to open and load the executable named `ls` relative to the process's current working directory. When the user is in the root directory `/`, `ls` succeeds because the binary is located at `/ls`. However, once inside directory `/d`, the kernel searches for `/d/ls`, which does not exist, causing `exec` to fail with `exec ls failed`.
   Way to run it anyway: We must supply the absolute path or relative path to the root binary:

   `/ls ..` or `../ls ..`

3. **Which missing feature would you miss most in daily work, and which would be easiest to add?**
   Missed most: The lack of `PATH` **lookup** and **command-line editing (Tab completion / history)**. Without `PATH`, organizing programs into clean directories like `/bin` or `/usr/bin` is impractical without typing full paths for every utility. Without Tab completion and history, every path and typo must be re-entered manually from scratch.
   Easiest to add: A basic `PATH` **lookup mechanism** or **simple quote stripping**. For `PATH`, in `user/sh.c`, inside `case EXEC:`, if `ecmd->argv[0]` does not begin with `/`, the shell could simply prefix `/` to the program name before invoking `exec()`. If the file exists in the root directory, it executes immediately without requiring the user to type `/ls`.

1.4 Part 3. Who expands *? A program that prints its argv

1.4.1 Utility Implementation for xv6: user/args.c

We write `user/args.c` to print the exact number of command-line arguments received and each individual string in `argv`:

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

We add `$U/_args\` to the `UPROGS` list in `Makefile`.

1.4.2 Utility Implementation for Linux: args_linux.c

We write the equivalent C utility for Linux adhering to POSIX standards:

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

We compile the program using gcc:

```
➜  shtest gcc -o args args_linux.c
```

1.4.3 Side-by-Side Execution

We run the exact same command lines in both shells:

```
# ------------------------------------------------------------ GNU bash (Linux)
➜  shtest ./args one    two    three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

➜  shtest ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

➜  shtest ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/ionut]

➜  shtest ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

```
# ------------------------------------------------------------ xv6 shell (QEMU)
$ args one    two    three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x y"
argc = 4
argv[0] = [args]
```

```
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]

$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Observe:

1. **Paste both outputs for args *.c "x y". How many arguments did each program receive?**
   - **GNU bash output:**

     ```
     argc = 3
     argv[0] = [./args]
     argv[1] = [args_linux.c]
     argv[2] = [x y]
     ```

     The Linux program received **3 arguments** (`argc = 3`).
   - **xv6 shell output:**

     ```
     argc = 4
     argv[0] = [args]
     argv[1] = [*.c]
     argv[2] = ["x]
     argv[3] = [y"]
     ```

     The xv6 program received **4 arguments** (`argc = 4`).
2. **So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?**

   The shell. This experiment provides conclusive proof.
   Neither the target binary (`args`) nor the kernel performs wildcard expansion or quotation stripping. In both Linux and xv6, the binary code of `args` does nothing more than iterate over whatever pointers are already populated in `argv`. The kernel's `exec` system call simply transfers the array of string pointers given to it into the newly loaded process's address space without altering their contents.
   In Linux, `bash` inspects the line before invoking `exec`, expands `*.c` against filesystem entries, strips the enclosing double quotes from `"x y"`, and treats `x y` as a single argument containing a space.
   In xv6, `user/sh.c` does not possess globbing or quotation-parsing logic; it merely tokenizes the line on whitespace. Therefore, `*.c`, `"x`, and `y"` are handed directly to the kernel and onto `args` as raw, unmodified strings.
3. **What happened to the extra spaces in one two three?**

   In both shells, consecutive spaces are treated strictly as **token delimiters**. When the shell's

lexical scanner encounters whitespace, it consumes all contiguous spaces until the next non-whitespace character is reached. The tokens (`one`, `two`, and `three`) are extracted and stored into distinct array slots in `argv`; the whitespace characters themselves are discarded and never passed to the invoked program.

1.5 Part 4. Read the shell

1.5.1 Source Code Inspection of user/sh.c

Inside the xv6 repository (`~/xv6-riscv`), we inspect the structure and implementation details of the shell:

```
➜  xv6-riscv git:(riscv) wc -l user/sh.c
499 user/sh.c

➜  xv6-riscv git:(riscv) grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10

➜  xv6-riscv git:(riscv) grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:

➜  xv6-riscv git:(riscv) grep -n "cd" user/sh.c | head -5
166:    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167:      // Chdir must be called by the parent, not the child.
168:      cmd[strlen(cmd) - 1] = 0; // chop \n
169:      if (chdir(cmd + 3) < 0)
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

1.5.2 Architecture of the xv6 Command Interpreter

The parser in `user/sh.c` models commands as an abstract syntax tree of polymorphic command structures sharing a common header (`struct cmd { int type; }`):

- `struct execcmd`: Holds program arguments (`argv`) for `exec()`.
- `struct redircmd`: Holds target file, flags, and file descriptor for redirection.
- `struct pipecmd`: Contains `left` and `right` subcommands connected via an anonymous pipe.
- `struct listcmd`: Contains subcommands executed in sequence (`;`).
- `struct backcmd`: Wraps a subcommand to be spawned asynchronously (`&`).

Observe:

1. **Name the five kinds of command the xv6 shell knows and the character that produces each.**

    1. `EXEC` (Type 1): An ordinary executable command; produced by regular word tokens delimited by whitespace.
    2. `REDIR` (Type 2): Stream redirection; produced by < (input redirect), > (output overwrite), and >> (output append, internal token +).
    3. `PIPE` (Type 3): Pipeline execution; produced by the pipe symbol `|`.
    4. `LIST` (Type 4): Sequential execution; produced by the semicolon `;`.
    5. `BACK` (Type 5): Asynchronous background execution; produced by the ampersand `&`.

2. **Where is cd handled, and why is it not run as a separate program?**

    Where it is handled: `cd` is handled directly inside the main loop of `main()` in `user/sh.c` (lines 166–170), preceding the call to `fork1()`:

    ```
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    }
    ```

    Why it cannot be an external program: In UNIX process architecture, every process maintains its own current working directory in kernel memory (in xv6, stored in `struct proc -> cwd`). A child process created via `fork()` receives a duplicate copy of the parent's `cwd`. If `cd` were an independent external binary (e.g., `/bin/cd`), the shell would fork a child process, the child would call `chdir()` to modify its own directory, and then exit. The parent shell process would remain unaffected in its original directory. Therefore, changing the shell's working directory **must** be performed by the parent process itself via a built-in command.

3. **Search sh.c for any code that handles * or $. What does that explain about Part 2?**

    Inspecting `user/sh.c` for `*` and `$` reveals that neither character appears anywhere in the tokenization table or parsing routines:

    ```
    char whitespace[] = " \t\r\n\v";
    char symbols[] = "<|>&;()";
    ```

    Because `*` and `$` are neither whitespace nor recognized syntax symbols, `gettoken()` treats them as standard alphanumeric characters (type `'a'`), packaging them directly into `argv`.

This confirms the empirical observations from Part 2 and Part 3: xv6 does not expand `ls *` because it lacks globbing logic, and it outputs `$HOME` literally because it has no variable lookup table.

2 Conclusion

Through this laboratory work, I investigated operating system concepts from the user-kernel interface, contrasting the production-grade GNU `bash` shell with the minimalist, pedagogical xv6 shell.

By fixing and verifying the `sleep` utility in xv6, I observed how user-space libraries and applications interact with evolving kernel system calls across ABI revisions. The empirical side-by-side comparison between `bash` and xv6 revealed the core responsibilities of a Unix shell: coordinating processes via `fork()`, managing execution images via `exec()`, synchronizing lifecycles via `wait()`, and orchestrating stream redirection via `pipe()` and `dup()`.

By developing and executing the `args` argument-inspection utility across both environments, I established a clear separation of concerns in Unix system architecture. The experiment proved that pathname wildcard expansion, environment variable substitution, and quote removal are purely user-space syntactic preprocessing duties handled by the shell, rather than services provided by the kernel or individual applications. Finally, dissecting `user/sh.c` demystified shell internals, demonstrating how an elegant 500-line C program leverages recursive descent parsing to construct an abstract syntax tree of commands that executes with remarkable efficiency.